

K8s (not-so-)gentle introduction

Francesco Faenza

SECloud research group
University of Modena and Reggio Emilia

DEP 2024/25



Main information

Open source orchestrator
Provides software for building
and deploying a reliable,
scalable distributed system
Velocity is a key concept in K8s
Core concepts are:

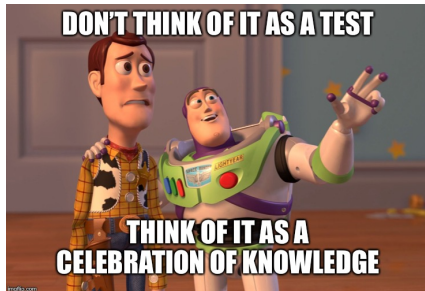
- immutability
- declarative configuration
- online self-healing system
- shared reusable libraries
and tools



Documentation: <https://kubernetes.io/docs/home/>

Best book: <https://www.manning.com/books/kubernetes-in-action-second-edition>
(now being updated, no need to buy all chapters, as they are available online)

orchestrator?
containerized app?
cloud-native app?
microservices app?



orchestrator: system that deploys and manages your applications (deploy it and dynamically respond to change)

in k8s: deploy, scale up and down, self-heal, perform zero-downtime updates

containerized app: app that runs in a container

cloud-native app: designed to meet cloud-like demands (auto-scale, self-heal, rolling updates, rollback, ...)

microservices app: a group of small, specialized, independent parts that work together to provide a meaningful application

AWS arrival was a hammer on other providers

Google needed a different focus to retain customers, the idea was to **abstract their cloud** value and make it easier for customers to approach

Gmail, Search, and co were already running with containers at scale. To orchestrate them, Google had **Borg** and **Omega**

They created **Kubernetes** and donated it to the **Cloud Native Computing Foundation (CNCF)** in 2014

While [Docker](#) and [Kubernetes](#) have always worked well together (they do different things, we'll see), Docker was quite “overkill” and k8s started working on a pluggable container runtime layer. In 2016, the [Container Runtime Interface \(CRI\)](#) was introduced

Now [Containerd](#) has replaced [Docker](#) as the default runtime
Containerized as Docker are fully compatible with the [Open Containers Initiative \(OCI\)](#)

In 2016/2017, [orchestrator wars](#) started. Swarm vs K8s vs others. Long story short, [k8s won](#)

Final touch to the history section

K8s “share DNA” with Borg and Omega (both Google proprietary)
Kubernetes is Greek for **Helmsman** (the person who steers a ship) -ζ the logo comes from it

In the beginning, the name should have been **Seven Of Nine** (Star Trek Borg drone), but you know, copyright

Still, **the seven spokes** of the logo are a subtle reference to this



K8s emerged as a **de facto platform** for deploying and managing cloud-native applications

In the same way Linux abstracts hardware differences in servers, Kubernetes could be used to **abstract differences between private and public clouds**

K8s allows for a **true hybrid cloud**

K8s is a cluster to run applications on and an orchestrator for cloud-native microservices apps

Cluster: a bunch of machines to host stuff. Each machine is a node. Each node can be whatever (physical, VM, Raspberry Pi, ...)

Orchestrator: fancy word for a system that takes care of deploying and managing
Main element of K8s cluster: **Control plane** nodes (cluster intelligence) and **Worker** nodes (cluster real work)

Control plane runs a collection of services to control the cluster effectively
One is enough in test/lab environments, but usually the minimum is 3, and recommended is 5 (obviously, depending on the size, complexity, and geographical distribution of the cluster, these values change)
Control plan **won't** (at least should not) **run user applications**

API server (RESTful API) is the main part. All communication between all components goes through the API server (all requests are subject to authentication and authorization)

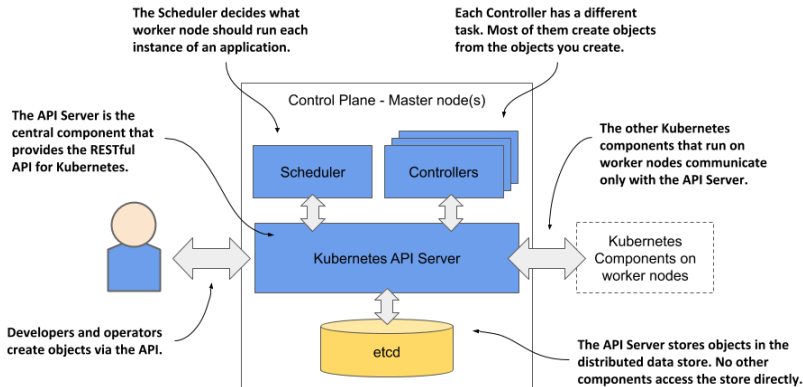
Cluster store, the stateful part of the control plane, persists the entire configuration and state of the cluster. Based on **etcd** (a popular distributed db). Being the single source of truth of the cluster and being replicated 3-5 times at least, etcd guarantees consistency over availability (uses **RAFT** consensus algorithm to accomplish this) [https://en.wikipedia.org/wiki/Raft_\(algorithm\)](https://en.wikipedia.org/wiki/Raft_(algorithm))

Controller manager, monitor the cluster, and respond to events. It is a controller of controllers because it spawns specific controllers for specific tasks (**Deployment controller**, **StatefulSet controller**, **ReplicaSet controller**, ...). The objective is to make sure that the observed state matches the **desired state**. (Obtain desired state, observe current state, determine differences, reconcile differences)

Scheduler, it watch API server for new tasks and assigns them to healthy workers nodes. (behind the scene complex logic to determine best nodes based on a ranking system). It runs predicate checks to determine task affinity or anti-affinity with task. If no node is able to run the task, the task remain pending.

Cloud controller manager, facilitate integration with cloud services, particularly with load-balancers, storages, and so on.

from: <https://livebook.manning.com/book/kubernetes-in-action-second-edition/chapter-1/v-14/124>



Workers nodes are where the application runs

Worker:

- watch the API server for a new work assignment
- execute the work
- report back to the control plane its status

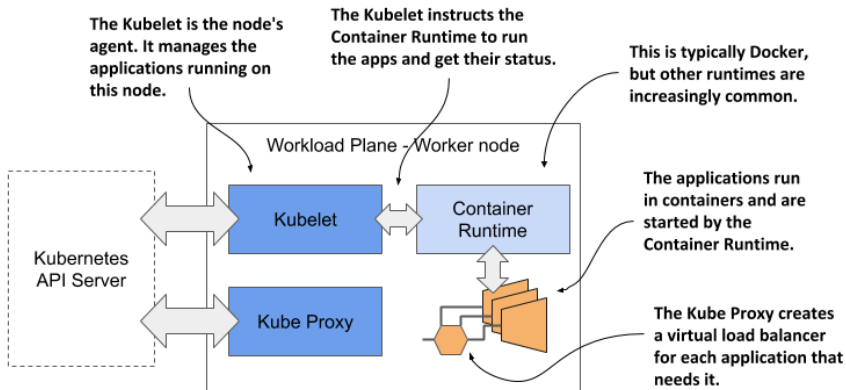
Kublet, the main K8s agent running on each worker node. As soon as it is installed, it reports node specs (CPU, RAM, storage, ...) to the control plane. It watches the API server for new work tasks and executes them, reporting back to the control plane.

Container runtime, used to perform container-related tasks (pulling images, starting and stopping containers, ...). It was Docker, now it is a plugin model called **Container Runtime Interface (CRI)**. (default containerd)

Kube-proxy, runs on every node and is responsible for local cluster networking. Ensures that each node gets its own IP and implements **iptables** or **IPVS** rules to handle routing and load-balancing. (<https://kubernetes.io/blog/2018/07/09/ipvs-based-in-cluster-load-balancing-deep-dive/>)

Worker nodes visual

from: <https://livebook.manning.com/book/kubernetes-in-action-second-edition/chapter-1/v-14/132>



Every cluster has a **DNS service** for service discovery
The DNS service has a static IP (hard-coded into every Pod) and is based on
CoreDNS (<https://coredns.io>)

To run an app in K8s, you'll need:

- a **Pod** (it will wrap your app)
- a **declarative manifest file** (it will deploy the app)

Write an app, build a container image out of it, store into a registry, define a Pod to run the containerized app, write a **Deployment** (a YAML manifest file that specify things like replicas, how to update, ...)

<https://livebook.manning.com/book/kubernetes-in-action-second-edition/chapter-2/v-14/1>

Declarative model and the concept of **desired state** are the heart of K8s
It works like this:

- declares the **desired state** of an app in a manifest file
- post to the API server
- K8s stores it into the cluster store as the application's **desired state**
- K8s implements the **desired state** on the cluster
- a controller makes sure the **observed state** fits the **desired state**

Pod is the **atomic unit** of K8s. Every container should run inside a pod.

Pod comes from the name of a group of whales

The meaning is that a Pod is not exactly a 1-to-1 relationship to containers. It can also run multiple containers (service meshes, web container with helper, container with logger, ...). You'll put together tightly coupled containers that may need to share memory and storage

Pods do not run applications, they run in containers. The Pod is the execution environment where one or more containers run (in the same Pod, they'll share network, volumes, IPC namespaces, shared memory, ...)

Pods are also the **minimum unit of scheduling** in k8s. When you scale, you do not add more containers, you add Pods

The Pod **deployment is an atomic operation**. So the Pod is ready when all its containers are up and running. Also, single Pod to single node, so all its containers are on the same node

Pods are **mortal**; if they die, a new one will start looking exactly like the old one (apart from ID and IP)

Pods are **immutable**; they don't change while running

Most Pods are being **indirectly** deployed by higher-level objects, most commonly **Deployments**, others are **DaemonSet** and **StatefulSet**

Deployment is an object that wraps around a **Pod** and adds features like self-healing, zero-downtime rollouts, and versioned rollbacks

Behind the scenes, **Deployments** (but also **DaemonSets** and **StatefulSets**) are controllers that constantly check that the observed state matches the desired one

Pods alone are unreliable (think about what changes when they are spawned)

Services provide reliable networking for a set of **Pods**

Behind the scenes, **Service** has **stable DNS name, IP address and port**. They load-balance traffic across **Pods**. As **Pods** come and go, the **Service** observes and updates itself

Service provides stable network abstraction (TCP and UDP)

Service does not possess application intelligence. Cannot provide application-layer host and path routing. For that, we'll need **Ingress**

Not simply a number of features you can ship per day/week/month

BUT

Number of things you can ship **while maintaining a highly available service**

Mutable system: current state of infrastructure != single artifact
rather than the sum of updates and changes to that artifact

Immutable system: entirely new image is built, an update replaces the entire
image in a single operation (no incremental changes)

i.e: mutable: `exec bash` in container, run a command to download something, kill
old service, and keep this

Immutable: build a new container image, push to registry, kill older, start newer
Immutable container images are the core of everything in K8s

Everything in k8s is a **declarative configuration object**

Alternative to **imperative**: state of something is the consequence of the execution of a defined series of instructions

Declarative: declaration of the desired state of a thing

K8s continuously ensures that the **current state matches the desired (declared) state**.

K8s initialize the system, then guard it from perturbations
(traditional way: manual intervention by someone)

A new paradigm - operator - with advanced logic to maintain, scale, and heal a specific piece of software (dbs, for example)

In **decoupled** architectures, each component is separated from other components by APIs and service load balancers

Communication overhead could be a problem

Scaling assumes that there are resources in the cluster to consume

Application decoupled from the machine by containers

K8s **automate scaling** (especially in a cloud environment with CLI machine creation)

Pods: a group of containers that constitute a single deployable unit

Services: provides load balancing, naming, and discovery to isolate microservices

ReplicaSet: main element in a K8s cluster, allows redundancy, scaling, and sharding of Pods

Ingress: an object that provides an easy-to-use frontend combining multiple microservices into a single externalized API surface area

principles: “not my monkey, not my circus”